

NIDO — Documentação Técnica para Investidores

NIDO — Documentação Técnica para Investidores

Trustless Escrow para Campanhas B2B sobre Stellar L1

Sumário

1. Executive Summary
 2. Produto e Tração
 - 2.1 O que a NIDO entrega
 - 2.2 Diferenciais e impacto gerado
 - 2.3 Público-alvo
 - 2.4 Tração
 3. Mainnet — Prova On-Chain
 - 3.1 Escrows reais financiados em mainnet
 - 3.2 Referência histórica — Validação em testnet (Sprint 1)
 4. Arquitetura e Fluxos Críticos
 - 4.1 Arquitetura de alto nível
 - 4.2 Ciclo de vida do escrow
 - 4.3 Fluxo 1 — Criação e funding (nativo Stellar)
 - 4.4 Fluxo 1b — Funding cross-chain (CCTP V2)
 - 4.5 Fluxo 2 — Liberação (Host aprova, Talent recebe)
 - 4.6 Fluxo 3 — Reembolso por expiração de prazo
 - 4.7 Fluxo 4 — Resolução de disputa (arbitragem neutra)
 5. Segurança e Modelo de Custódia
 6. Modelo de Negócio e GTM
 - 6.1 Modelo de receita
 - 6.2 Canais de aquisição
 - 6.3 Síntese de posicionamento
 7. Roadmap e Investimento Pleiteado
 - 7.1 Status dos Sprints (37 Graus, Track A)
 - 7.2 Desafios “Deploy Mainnet”
 - 7.3 Investimento pleiteado
 8. Riscos e Mitigações
- Anexo A — Decisões de Produto (PDRs)
- Anexo B — Decisões de Arquitetura (ADRs)
- Anexo C — Referência de Endpoints e Configuração Mainnet
- C.1 Endpoints Stellar (todos sob authGuard e rate limit por usuário)
 - C.2 Configuração Mainnet (valores públicos)
 - C.3 Contratos CCTP (Soroban, domínio 27)
 - C.4 Stack tecnológico

NIDO — Documentação Técnica para Investidores

Trustless Escrow para Campanhas B2B sobre Stellar L1

Versão 2.1 — 2026-05-30 · Programa 37 Graus (NearX/SDF), Track A · Stellar Mainnet (produção) Builder: Gustavo Fontes (f0ntz) · fontzweb3@gmail.com · Brazil Chapter (Lead: Caio Mattos) Repositórios:
github.com/Paradevs1/nido-api (backend) ·
github.com/Paradevs1/nido-front (frontend)

Sumário

- [1. Executive Summary](#)
 - [2. Produto e Tração](#)
 - [3. Mainnet — Prova On-Chain](#)
 - [4. Arquitetura e Fluxos Críticos](#)
 - [5. Segurança e Modelo de Custódia](#)
 - [6. Modelo de Negócio e GTM](#)
 - [7. Roadmap e Investimento Pleiteado](#)
 - [8. Riscos e Mitigações](#)
 - [Anexo A — Decisões de Produto \(PDRs\)](#)
 - [Anexo B — Decisões de Arquitetura \(ADRs\)](#)
 - [Anexo C — Referência de Endpoints e Configuração Mainnet](#)
-

1. Executive Summary

A NIDO é uma plataforma B2B de campanhas de marketing com escrow trustless sobre Stellar L1. Conecta marcas e agências (*Hosts*) a criadores de conteúdo e KOLs (*Talents*), transformando contratos de campanha em acordos on-chain invioláveis — sem que nenhuma das partes precise manter saldo de XLM, pagar taxa de rede ou confiar a custódia dos fundos a um terceiro.

Problema. O mercado de marketing de influência sofre com quebra de acordo dos dois lados: agências imobilizam capital com risco de não-entrega; criadores entregam e não recebem. As soluções on-chain existentes (contratos EVM customizados) endereçam a confiança, mas com taxa de rede elevada, complexidade operacional e atrito cripto que o cliente B2B Web2-first não absorve.

Solução. A NIDO usa primitivas determinísticas da L1 da Stellar — Multisig 2-de-3, Sponsored Reserves, transações pré-autorizadas com TimeBounds e FeeBump — para construir um cofre trustless sem escrever nenhum smart contract customizado, com finalidade em aproximadamente 5 segundos e custo de rede inferior a US\$ 0,01 por transação. A experiência do usuário é equivalente à de um SaaS Web2: a camada blockchain permanece transparente para o cliente.

Estado em 30/05/2026. O produto opera em mainnet. O ciclo completo de escrow está implementado e testado — criação, funding (nativo e cross-chain via CCTP V2), liberação, reembolso por expiração de prazo e resolução de disputa por árbitro neutro. Backend com 409 testes automatizados em 23 suites (todos passando), rate limiting por usuário e chaves de escrow encriptadas em repouso (AES-256-GCM).

Categoria	Plataforma B2B SaaS + infraestrutura de escrow trustless (non-custodial)
Blockchain	Stellar L1 (mainnet) + Soroban (CCTP inbound, domínio 27)
Estágio	Produto em produção (beta público) e em operação na mainnet Stellar · Track A 37 Graus
Tração da plataforma	+500 usuários · 300+ creators · 200+ KOLs · marcas reais como Host (JET Latam, CoinW, Venus, MEXC) · payouts em USDC
Tração na mainnet Stellar	5 escrows reais financiados (50 USDC); funding, liberação e disputa comprovados on-chain (§3.1)
Modelo de receita	Planos SaaS (BASIC/CORE/ENTERPRISE) + fee de 0,5% sobre o volume liquidado em escrow
Investimento pleiteado	SCF Build Award — aproximadamente US\$ 70 mil para escalar a operação em mainnet (§7.3)

2. Produto e Tração

2.1 O que a NIDO entrega

Plataforma de campanhas B2B construída sobre três pilares de escrow trustless. Cada pilar endereça uma fricção concreta que hoje limita a adoção de cripto por agências e marcas.

1. Escrow trustless com Sponsored Reserves. O Treasury da NIDO patrocina 100% das reserves da conta de escrow; a conta existe no ledger com 0 XLM próprio e o Host deposita apenas USDC. O Multisig 2-de-3 garante que nem a plataforma, nem o Host, nem o Talent movimentem fundos isoladamente.

Impacto: o capital da campanha fica imobilizado de forma neutra e auditável. A NIDO não detém os fundos em nenhum momento, o que a mantém fora do perímetro de custodiante/transmissor de dinheiro, reduz exposição regulatória e remove a desconfiança que leva uma agência a hesitar em adiantar pagamento.

2. Experiência sem taxa de rede para o usuário. A conta de escrow tem 0 XLM; o Treasury usa FeeBump para arcar com a taxa de rede de toda transação (funding, liberação, reembolso e disputa). Host e Talent nunca precisam manter saldo de moeda

nativa.

Impacto: a camada blockchain torna-se transparente. O cliente B2B Web2-first não precisa lidar com XLM, gas ou bridge — a principal barreira de adoção cripto é eliminada, permitindo um ciclo comercial convencional, sem etapa de educação técnica no onboarding.

3. Proteção contra não-entrega com TimeBounds. Transações pré-autorizadas com TimeBounds criam um reembolso automático disponível após o prazo da campanha. Se o Talent não entrega, o Host recupera o capital — o protocolo Stellar valida o minTime on-chain, sem dependência de a NIDO estar operante.

Impacto: o capital do Host é protegido pelo protocolo, não por uma garantia contratual da plataforma. O reembolso permanece executável mesmo em caso de indisponibilidade da NIDO, deslocando a confiança da plataforma para as regras determinísticas da L1.

2.2 Diferenciais e impacto gerado

Capacidade	Impacto gerado
Zero XLM para o usuário (Sponsored Reserves + FeeBump)	Onboarding sem atrito cripto, reduzindo abandono e encurtando o ciclo de venda B2B.
Trustless por design (Multisig 2-de-3)	NIDO como árbitro neutro, nunca custodiante — menor exposição regulatória e maior disposição do cliente a adiantar capital.
Non-custodial (chaves nunca tocam o backend)	Um eventual comprometimento do servidor não movimenta fundos — superfície de ataque substancialmente menor e <i>due diligence</i> mais simples.
Funding cross-chain (CCTP V2, 6 chains)	O Host financia a partir da chain em que já mantém USDC (Ethereum, Arbitrum, Base, Polygon, Solana, Sui), removendo o atrito de aporte e ampliando o funil.
Coexistência multichain (módulo Stellar isolado)	A integração Stellar não altera os rails legados (EVM/Solana/Sui) — sem risco de regressão, com rollout incremental sobre uma base já em produção.
Finalidade em ~5s e taxa de rede inferior a US\$ 0,01 (Stellar L1)	Liquidação quase instantânea ao Talent, sem erosão de margem por gas, viabilizando inclusive campanhas de ticket baixo.

2.3 Público-alvo

Segmento	Dor central	Solução da NIDO
Agências e iniciativas regionais (ex.: JET Latam, Host ativo)	Capital em risco: quebra de acordo antes ou após a entrega	Capital imobilizado de forma neutra e reembolso automático por prazo,

Segmento	Dor central	Solução da NIDO
Marcas e exchanges B2B (ex.: CoinW, Venus, MEXC, Hosts ativos)	Pagamentos internacionais lentos, custosos e sem rastreabilidade	permitindo adiantar pagamento com segurança Liquidação em USDC em segundos, auditável on-chain — tesouraria previsível e internacional
KOLs e criadores	Não-recebimento após a entrega; pagamentos com dias de atraso	Garantia de que o USDC já está imobilizado antes da produção; recebimento na aprovação
Projetos Web3	Custo de contratos customizados; necessidade de infraestrutura pronta	Escrow trustless como serviço, sem desenvolvimento de contrato próprio — adoção imediata

2.4 Tração

A tração é reportada em dois blocos distintos — a plataforma (em produção, com marcas e creators reais) e o módulo Stellar (recém-lançado, com prova on-chain em mainnet). Os blocos não são consolidados, para preservar a precisão de cada indicador.

Bloco A — Plataforma NIDO (em produção, beta público)

A plataforma opera em produção em escala: marcas reais publicam campanhas e creators recebem USDC por meio dos rails multichain existentes.

Indicador	Valor
Usuários	+500
Creators ativos	300+
KOLs alcançados	200+
Marcas com campanhas ativas (Hosts)	JET Latam, CoinW, Venus Protocol, MEXC, Rapidz, BH OnChain, TokenNation
Payouts a creators (USDC)	Recorrentes — maiores recebimentos: US\$ 500, US\$ 280, US\$ 271, US\$ 210, US\$ 186
Modelo de receita	Planos SaaS ativos + fee sobre volume

A NIDO não é um MVP em busca de mercado: é um produto em produção, com demanda B2B validada e marcas pagando creators. O escrow trustless sobre Stellar entra como upgrade de infraestrutura sobre uma operação existente, e não como tese especulativa.

Bloco B — Módulo de Escrow Trustless (Stellar mainnet)

Os indicadores abaixo referem-se exclusivamente ao novo fluxo de escrow trustless sobre Stellar — as primeiras execuções reais desse módulo em mainnet. Não representam a operação total da NIDO (Bloco A); constituem a prova de que a camada de custódia trustless funciona end-to-end com USDC real.

Indicador	Valor	Prova
Escrows financiados em mainnet	5 (4 liberados + 1 reembolsado via disputa)	Hashes on-chain (§3.1)
Volume USDC no fluxo de escrow	50 USDC	StellarExpert
Carteiras distintas no fluxo de escrow	5 Talents + 1 Host	On-chain
Fluxos comprovados on-chain	Funding, liberação e resolução de disputa	§3.1
Testes automatizados (backend)	409 testes em 23 suites (todos passando)	<code>npx jest --runInBand</code>

3. Mainnet — Prova On-Chain

O produto opera na mainnet pública da Stellar. Configuração verificada via script de health-check (`npm run stellar:check`) em 2026-05-30:

Item	Valor (mainnet)
Rede	STELLAR_NETWORK=mainnet
Horizon	https://horizon.stellar.org
Soroban RPC (CCTP)	https://mainnet.sorobanrpc.com
USDC Issuer (Circle)	GA5ZSEJYB37JRC5AVCIA5M0P4RHTM335X2KGX3IH0JAPP5RE34K4KZVN
Treasury (sponsor e fee payer)	GBG7UXZDEK3QG5Y3KXGIR7AB2CJQVR3DI2JY34TV2IFT0Y67LAUBVA4T
Arbiter (árbitro neutro)	GC27X2AXTFFF7UNDMBNQ55XVKRRTN07JPLUELEMPJJDQDA57G5TW3JUE
Encriptação de chave de escrow	AES-256-GCM (round-trip validado)

3.1 Escrows reais financiados em mainnet

Em 2026-05-30, 5 escrows foram financiados com USDC real e executados em ciclo completo na mainnet pública da Stellar, totalizando 50 USDC. Cada escrow percorreu o lifecycle completo: depósito do Host, cofre Multisig 2-de-3 (0 XLM próprio, reserves

patrocinadas), liberação para o Talent ou reembolso via arbitragem, e `accountMerge` devolvendo as reservas ao Treasury. Todas as transações foram bem-sucedidas e são auditáveis on-chain (ledgers 62.805.690 a 62.806.460).

Cobertura dos fluxos críticos em mainnet:

- 4 escrows liberados (COMPLETED): Host aprovou e Talent recebeu o USDC. Liberação validada — o Talent GAABSLGD...NHXNJ mantém 10 USDC em saldo.
- 1 escrow reembolsado via disputa (REFUNDED): Host abriu disputa, o árbitro resolveu a favor do Host e o USDC foi devolvido. Caminho de arbitragem comprovado em mainnet.

O reembolso por expiração de prazo (Fluxo 3) utiliza a mesma primitiva de pagamento ao Host já exercida no caso de disputa; sua automação está implementada e coberta por testes, ainda sem um evento de expiração isolado em mainnet.

Slot	Conta de escrow	Valor	Status final	Prova on-chain
1	GAAYTBS4... HC7Z	10 USDC	Liberado (Talent recebeu)	StellarExpert
2	GDEKEYRT... EBDB	10 USDC	Liberado (Talent recebeu)	StellarExpert
3	GA7YFF32... CRK3	10 USDC	Liberado (Talent recebeu)	StellarExpert
4	GDBG67YU... WCE2	10 USDC	Liberado (Talent recebeu)	StellarExpert
5	GBEKWXML... XWTK	10 USDC	Reembolsado (disputa, Host)	StellarExpert

Cada escrow é encerrado com `accountMerge`, devolvendo as reservas patrocinadas ao Treasury (GBG7UXZD...BVA4T), verificável no [StellarExpert](#). Os hashes de funding completos estão disponíveis no banco para auditoria.

Nota metodológica: os 5 escrows acima compartilham o mesmo Host, caracterizando validação operacional do fluxo em mainnet. A ampliação para usuários externos independentes está em andamento (ver §7.2, Desafio 3).

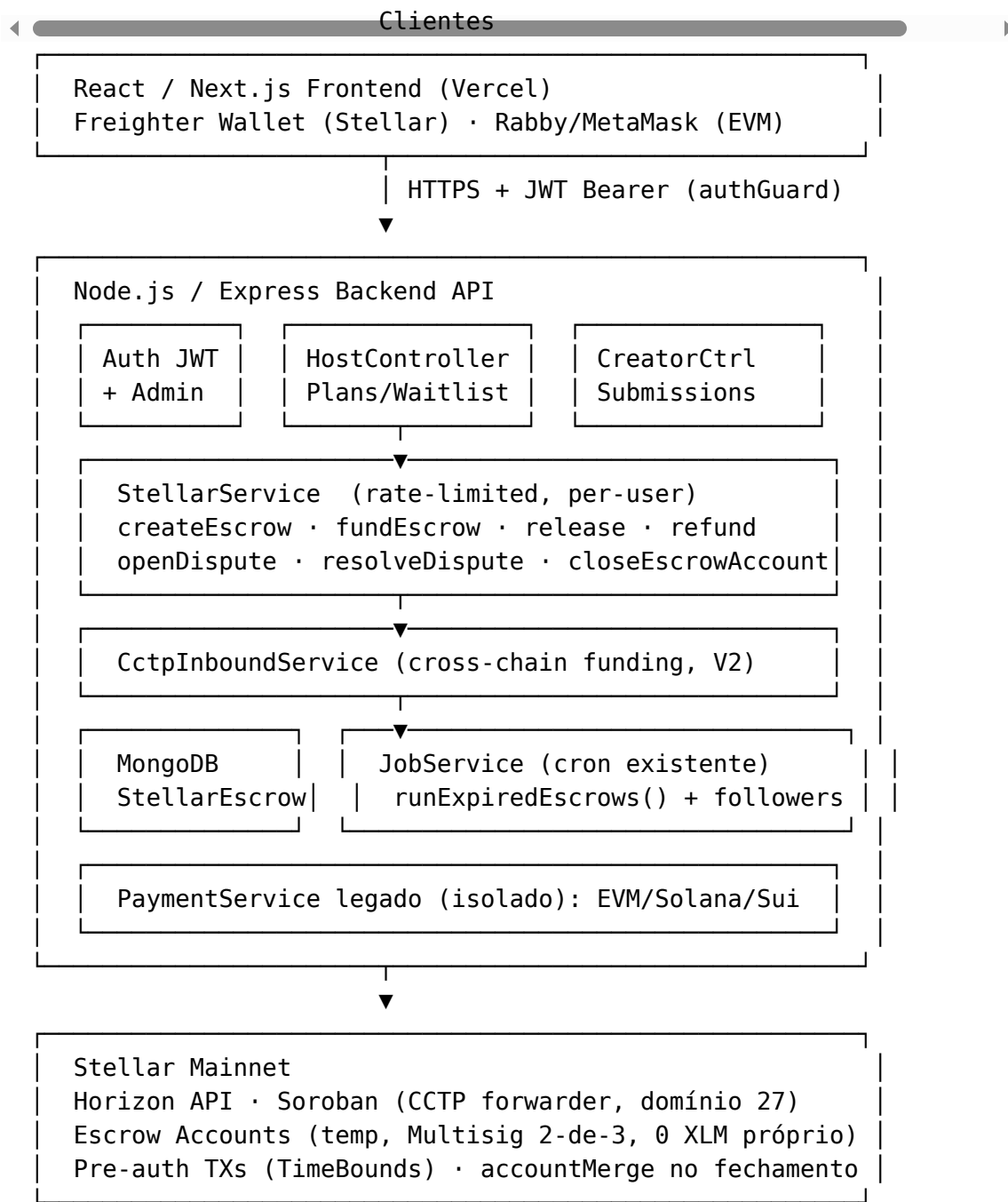
3.2 Referência histórica — Validação em testnet (Sprint 1)

Item	Valor
Conta de escrow	GBKZYR42QJHPQWHKCZG3ZY4CGWC65WSR7VARULZRXS4I7MBQ6JVGGYJL
TX Hash	aa9f3302a053b87c56332a88e26dfa7bc903466795e75a1af354467340
USDC imobilizado	10 USDC, 6 reservas patrocinadas (0 XLM próprio)

Item	Valor
Multisig	Host W=1, Talent W=1, Arbiter W=1 (Med=2, High=2)

4. Arquitetura e Fluxos Críticos

4.1 Arquitetura de alto nível



4.2 Ciclo de vida do escrow

CREATED → PENDING_INBOUND_MINT¹ → FUNDED → { COMPLETED | REFUNDED | DISPUTED }

¹ PENDING_INBOUND_MINT aplica-se apenas ao funding via CCTP (cross-chain), enquanto se aguarda a atestação da Circle Iris e o mint do forwarder Soroban. O funding nativo Stellar transita diretamente de CREATED para FUNDED.

4.3 Fluxo 1 — Criação e funding (nativo Stellar)

1. Host: POST /api/stellar/escrow/create
{ job_id, host_wallet, talent_wallet, amount, deadline }
2. Backend (StellarService.createEscrow):
 - gera keypair temporária do escrow (chave encriptada AES-256-GCM em repouso)
 - beginSponsoringFutureReserves -> createAccount(startingBalance:'0')
 - changeTrust(USDC) -> setOptions x3 (Host/Talent/Arbiter, W=1)
 - setOptions(thresholds: med=2, high=2)
 - endSponsoringFutureReserves
 - buildPreAuthTransactions: payment_xdr (->Talent) + refund_xdr (->Host, minTime=deadline)
 - persiste StellarEscrow { status: CREATED }
3. Host: GET /api/stellar/escrow/:jobId/funding-xdr -> XDR unsigned do depósito
4. Host: Freightier.sign(funding_xdr)
5. Host: POST /api/stellar/escrow/fund { host_signed_xdr }
 - Backend envelopa em FeeBumpTransaction (Treasury arca com a taxa)-> submete à Horizon
 - status: FUNDED. USDC imobilizado. Escrow: 0 XLM próprio, reserves patrocinadas.

4.4 Fluxo 1b — Funding cross-chain (CCTP V2)

1. Host: POST /api/stellar/escrow/inbound/prepare { source_chain, amount }
 - backend retorna instruções de burn (depositForBurnWithHook na chain de origem)
2. Host queima USDC na chain de origem (Ethereum/Arbitrum/Base/Polygon/Solana/Sui)
3. Host: POST /api/stellar/escrow/inbound/register { burn_tx_hash }
 - status: PENDING_INBOUND_MINT
4. Backend aguarda atestação da Circle Iris (V2)
5. POST /api/stellar/escrow/inbound/relay (permissionless)
 - chama mint_and_forward(message, attestation) no forwarder Soroban (domínio 27)
 - USDC cunhado diretamente na conta de escrow -> status: FUNDED

4.5 Fluxo 2 — Liberação (Host aprova, Talent recebe)

Talent entrega -> Host revisa e aprova

1. GET /api/stellar/escrow/:jobId/payment-xdr -> XDR unsigned (escrow -> Talent)
2. Host: Freightier.sign(payment_xdr)
3. POST /api/stellar/escrow/release { host_signed_xdr }
 - backend adiciona a assinatura do Arbiter (fecha o threshold 2-de-3)
 - FeeBumpTransaction (Treasury arca com a taxa; escrow tem 0 XLM)
 - submete à Horizon -> Talent recebe USDC em menos de 5s
 - closeEscrowAccount: accountMerge devolve o XLM patrocinado ao Treasury
 - status: COMPLETED

4.6 Fluxo 3 — Reembolso por expiração de prazo

JobService.runExpiredEscrows() (cron):

busca StellarEscrow { status: FUNDED, deadline < now }
verifica on-chain via Horizon -> notifica o Host (TimeBound de reembolso ativo)

Host solicita reembolso:

GET /api/stellar/escrow/:jobId/refund-xdr -> Host assina (Freightier)
POST /api/stellar/escrow/refund { host_signed_xdr }

- o protocolo Stellar valida o minTime (TimeBound) on-chain
- Arbiter assina -> threshold 2 -> FeeBump -> submete
- accountMerge -> status: REFUNDED. Host recupera o USDC.

4.7 Fluxo 4 — Resolução de disputa (arbitragem neutra)

Host recusa a entrega; Talent discorda -> disputa

1. POST /api/stellar/dispute { job_id, reason, initiator } -> status: DISPUTED
 2. GET /api/stellar/admin/disputes (adminGuard) -> árbitro lista as disputas
 3. Árbitro revisa evidências -> POST /api/stellar/admin/resolve { winner: HOST|TALENT }
 - backend assina (Arbiter) o XDR vencedor e persiste dispute_resolution_xdr
 4. Parte vencedora: GET /escrow/:jobId/dispute-xdr -> assina (Freightier)
 5. POST /api/stellar/dispute/claim { winner_signed_xdr }
 - threshold 2-de-3 fechado -> FeeBump -> submete -> accountMerge
 - USDC transferido à parte vencedora; log auditável no MongoDB
-

5. Segurança e Modelo de Custódia

A segurança não é uma camada acessória, e sim o núcleo do produto. Toda a operação foi desenhada sobre um princípio: abstrair a complexidade cripto do usuário sem nunca assumir o controle dos seus fundos. A tabela abaixo associa cada mecanismo ao risco concreto que ele elimina.

Mecanismo	Risco eliminado
Custódia — Multisig 2-de-3 (Host + Talent + Arbiter, W=1 cada; threshold 2)	Remove a NIDO do papel de custodiante: nenhuma parte movimenta fundos isoladamente, eliminando ponto único de comprometimento e o enquadramento como transmissor de dinheiro.
Chaves do usuário — assinatura via Freightier (nunca tocam o backend)	Um vazamento do servidor não concede acesso aos fundos do usuário: a chave privada nunca está sob responsabilidade da plataforma (ADR-002).
Chave do escrow — encriptada em repouso (secret_key_encrypted, AES-256-GCM; masterWeight sem chave operável)	A conta de escrow é inoperável fora do multisig: mesmo com acesso ao banco, não há como destravar o cofre.
Reembolso — pré-autorização + TimeBounds (minTime = deadline, validado on-chain)	Capital do Host protegido pelo protocolo: o reembolso permanece executável mesmo com a NIDO indisponível (ADR-004).
Autenticação — JWT + adminGuard (todas as rotas; árbitro isolado)	Superfície administrativa segregada da de usuário: a arbitragem não é acessível por contas comuns.
Rate limiting por usuário (chave = userId do JWT, não IP)	Protege o Treasury de exaustão de reserves e mitiga abuso, sem penalizar usuários que compartilham um mesmo IP.
Fee sponsorship — FeeBump via Treasury	A conta de escrow opera com 0 XLM e o usuário não paga taxa de rede: é a abstração que viabiliza a experiência Web2 (ADR-003).
Consistência — verificação on-chain via Horizon (antes de toda liberação/reembolso)	O banco nunca é fonte de verdade sobre fundos: não há liberação baseada em estado local divergente do ledger.
CCTP relay permissionless (mint_and_forward, finalidade standard)	Funding cross-chain sem custódia intermediária nem bridge: USDC nativo de ponta a ponta, sem risco de wrapped token.

Hardening previsto para escala (roadmap): migração da chave do Arbiter para KMS/HSM (atualmente encriptada em ambiente isolado), Horizon privada com SLA e alerta automático de saldo do Treasury abaixo de 20%.

6. Modelo de Negócio e GTM

6.1 Modelo de receita

Fonte	Mecanismo	Status
Fee de escrow	0,5% sobre o volume USDC liquidado via Stellar	Ativo em mainnet
Planos SaaS	BASIC / CORE / ENTERPRISE (ativos nos rails legados)	Em produção
Fee de campanha	Taxa fixa por campanha com pagamento via Stellar	Mainnet

Sobre micropagamento (x402): o fluxo da NIDO é humano-para-humano (o Host avalia subjetivamente a entrega do Talent) e o cliente B2B opera com contratos, faturas e assinaturas — um débito por requisição é incompatível com esse ciclo comercial. O x402 é considerado um vetor futuro (exposição de API paga para automação agentic), fora do escopo atual.

6.2 Canais de aquisição

Canal	Tática	Meta
Base de Hosts ativos	Migrar marcas que já rodam campanhas (JET Latam, CoinW, Venus, MEXC) para o fluxo de escrow trustless	Converter demanda existente em volume on-chain
Base de creators (300+)	Creators como primeiros usuários do escrow Stellar (dogfooding)	Liquidez de oferta no fluxo trustless
Comunicação pública (X)	Publicação periódica de métricas com transações on-chain como prova	Credibilidade e awareness
Comunidade Stellar BR	Presença em Telegram e Discord	Awareness
Rede 37 Graus	Mentores SDF e introduções B2B no Rio	Pipeline de leads qualificados

6.3 Síntese de posicionamento

Configure um escrow de campanha em poucos minutos. O criador recebe USDC automaticamente quando a entrega é aprovada; se o prazo expira sem entrega, o Host recupera o capital de forma automática e sem depender de terceiros. Sem gas e sem XLM para o usuário, liquidação auditável on-chain e internacional em segundos.

7. Roadmap e Investimento Pleiteado

7.1 Status dos Sprints (37 Graus, Track A)

Sprint	Período	Status	Entregáveis-chave
1 — Foundation	04–11/05	Concluído	StellarService; Sponsored Reserves + Multisig 2-de-3 + FeeBump; escrow em testnet
2 — Global Payments	11–18/05	Concluído	Liberação + reembolso; integração Freighter; UI do Host
3 — Dispute & B2B	18–25/05	Concluído	Disputa + resolução admin; runExpiredEscrows; accountMerge; UI Talent/Arbiter
4 — Scale & Mainnet	25/05–01/06	Em curso	Mainnet em produção; 5 escrows reais (50 USDC); CCTP inbound; documentação de investidor
5 — Stellar Village	08–11/06	Planejado	Pitch presencial (Rio); demonstração ao vivo

7.2 Desafios “Deploy Mainnet”

Desafio	Descrição	Status
1 — Produto ao vivo na mainnet	Publicar e comprovar o produto na mainnet Stellar	Concluído — 5 escrows reais on-chain (§3.1)
2 — Materiais de apresentação	Pitch deck, landing page e documentação técnica	Em curso — este documento
3 — 5 usuários reais na mainnet	5 carteiras ativas em mainnet	Em curso — 5 Talents distintos com escrow real (§3.1); validação com usuários externos independentes em andamento
4 — GTM e logística do Village	Plano de crescimento, viagem e aplicação SCF	Em curso — viagem ao Rio confirmada; aplicação SCF a submeter

7.3 Investimento pleiteado

Aplicação ao SCF Build Award (Stellar Community Fund), no montante aproximado de US\$ 70 mil, para escalar a operação de escrow trustless em mainnet. Valor e objetivo detalhados em documento SCF dedicado; submissão pendente.

Uso de recursos previsto:

- Engenharia: hardening de produção e escala do módulo de escrow.
- Segurança: migração da chave do Arbiter para KMS/HSM e alerta automático de saldo do Treasury.
- Infraestrutura: Horizon privada com SLA e Soroban RPC dedicado (CCTP).
- Comercial: conversão da base de Hosts ativos (JET Latam, CoinW, Venus, MEXC) para o fluxo de escrow trustless.

8. Riscos e Mitigações

Risco	Probabilidade	Impacto	Mitigação
Esgotamento de XLM do Treasury (sponsor e taxas)	Baixa	Alto	Alerta abaixo de 20%; reposição imediata; rate limit anti-burst na criação
Resistência de clientes B2B ao Freighter	Alta	Alto	Onboarding dedicado; CCTP permite financiar a partir de chain familiar
Chave do Arbiter fora de KMS/HSM	Baixa	Crítico	Encriptada em ambiente isolado; migração para KMS no roadmap
Pré-autorização inválida após mudança de estado on-chain	Baixa	Alto	Verificação on-chain via Horizon antes de toda liberação/reembolso
Rate limit da Horizon pública	Baixa	Médio	Retry com backoff; avaliação de Horizon privada com SLA
Atestação da Iris (CCTP) lenta ou com falha	Média	Médio	Funding nativo Stellar como caminho padrão; CCTP é opcional
Conversão lenta dos Hosts ativos para o escrow	Média	Médio	Base de marcas já em produção reduz o risco de aquisição; o foco é

Risco	Probabilidade	Impacto	Mitigação
			migração, não prospecção do zero

Anexo A — Decisões de Produto (PDRs)

PDR-001 — Multisig 2-de-3 com Sponsored Reserves como mecanismo de escrow. Conta temporária criada com `beginSponsoringFutureReserves` (0 XLM), `setOptions` com `thresholds med/high = 2` e três signatários externos `W=1` (Host/Talent/Arbiter). Keypair encriptado após o setup. *Alternativa rejeitada:* smart contract Soroban customizado para custódia. *Justificativa:* primitivas L1 são determinísticas, auditáveis e suficientes, com menor superfície de ataque que um contrato customizado.

PDR-002 — Sponsored Reserves + FeeBump para experiência sem atrito. Escrow criado com `startingBalance: '0'`; o Treasury envelope as taxas em FeeBump. *Justificativa:* clientes B2B são Web2-first; qualquer exigência de XLM é barreira de adoção. Custo por escrow inferior a US\$ 0,01.

PDR-003 — TimeBounds + pré-autorização para proteção contra não-entrega. Dois XDRs pré-autorizados: `payment_xdr` (Talent, imediato) e `refund_xdr` (Host, `minTime=deadline`). *Justificativa:* validados on-chain; o Host recupera o capital mesmo se a NIDO ficar indisponível.

PDR-004 — MongoDB com extensão de schema (não disruptiva). Coleção `stellar_escrows` e campos opcionais, sem migração de banco. *Justificativa:* plataforma em produção; migração seria risco desnecessário. Consultas sobre fundos sempre verificam o estado on-chain.

PDR-005 — Coexistência com a infraestrutura multichain legada. Rotas `/api/stellar/*` totalmente isoladas do `PaymentService` (EVM/Solana/Sui). *Justificativa:* rollout incremental, sem risco de regressão.

PDR-006 — Funding cross-chain via CCTP V2 (não bridge). O Host pode financiar a partir de 6 chains queimando USDC nativo; o mint na conta de escrow ocorre via forwarder Soroban. *Justificativa:* USDC nativo de ponta a ponta (sem wrapped token, sem risco de bridge), ampliando o funil sem atrito.

PDR-007 — Track A no 37 Graus. Produto com MVP em operação; a integração Stellar é expansão. Classificação coerente com o estágio real e competitiva nos critérios de escala.

Anexo B — Decisões de Arquitetura (ADRs)

ADR-001 — StellarService como camada de abstração de chain. Encapsula toda a interação com a Stellar SDK; os controllers roteiam por `payment_chain`. *Rejeitado:* lógica `if (chain === 'stellar')` dispersa nos controllers, por acoplamento e impossibilidade de teste isolado.

ADR-002 — XDR gerado no backend, assinado no frontend (non-custodial). O backend monta o XDR unsigned, o Freighter assina, e o backend aplica FeeBump e submete. As chaves do usuário nunca tocam o servidor.

ADR-003 — Sponsored Reserves + FeeBump via keypair do Treasury. `beginSponsoringFutureReserves + createAccount(startingBalance: '0');` o Treasury assina o FeeBump de toda transação subsequente.

ADR-004 — Pré-autorização para liberação/reembolso/disputa sem custódia. XDRs completos persistidos no MongoDB; a execução requer apenas o fechamento do threshold 2-de-3. A NIDO nunca detém controle unilateral.

ADR-005 — JobService existente como worker de TimeBounds. `runExpiredEscrows()` adicionado ao cron existente: detecta prazos vencidos, verifica on-chain e notifica o Host. Sem novo processo a manter.

ADR-006 — MongoDB com StellarEscrow e accountMerge no fechamento. `closeEscrowAccount` executa `accountMerge` na liberação/reembolso/disputa, devolvendo o XLM patrocinado ao Treasury.

ADR-007 — CctpInboundService para funding cross-chain. Serviço dedicado orquestra burn, atestação da Iris e `mint_and_forward` no forwarder Soroban (domínio 27). Relay permissionless; finalidade standard por padrão.

Anexo C — Referência de Endpoints e Configuração Mainnet

C.1 Endpoints Stellar (todos sob authGuard e rate limit por usuário)

Endpoint	Método	Responsabilidade
<code>/api/stellar/escrow/create</code>	POST	Cria o setup do escrow (sponsored + multisig + pré-autorização)
<code>/api/stellar/escrow/:jobId/funding-xdr</code>	GET	XDR unsigned do depósito (Host assina)
<code>/api/stellar/escrow/fund</code>	POST	Submete o funding assinado (FeeBump)
<code>/api/stellar/escrow/inbound/prepare</code>	POST	Prepara o funding CCTP (instruções de burn)
<code>/api/stellar/escrow/inbound/register</code>	POST	Registra a burn tx da chain de origem
<code>/api/stellar/escrow/inbound/relay</code>	POST	<code>mint_and_forward</code> no forwarder Soroban
<code>/api/stellar/escrow/:jobId/payment-xdr</code>	GET	XDR unsigned da liberação (Talent)

Endpoint	Método	Responsabilidade
/api/stellar/escrow/:jobId/refund-xdr	GET	XDR unsigned do reembolso (TimeBound)
/api/stellar/escrow/release	POST	Host aprova; backend fecha o threshold e submete
/api/stellar/escrow/refund	POST	Reembolso após expiração do prazo
/api/stellar/escrow/:jobId/status	GET	Status em tempo real via Horizon
/api/stellar/dispute	POST	Abre disputa (evidência e initiator)
/api/stellar/escrow/:jobId/dispute-xdr	GET	XDR de resolução para a parte vencedora
/api/stellar/dispute/claim	POST	Parte vencedora reivindica os fundos
/api/stellar/admin/disputes	GET	(adminGuard) Lista as disputas abertas
/api/stellar/admin/resolve	POST	(adminGuard) Árbitro resolve (winner: HOST/TALENT)

C.2 Configuração Mainnet (valores públicos)

STELLAR_NETWORK = mainnet
 Horizon = https://horizon.stellar.org
 Soroban RPC = https://mainnet.sorobanrpc.com
 USDC Issuer (Circle) =
 GA5ZSEJYB37JRC5AVCIA5MOP4RHTM335X2KGX3IH0JAPP5RE34K4KZVN
 Treasury (público) =
 GBG7UXZDEK3QG5Y3KXGIR7AB2CJKVR3DI2JY34TV2IFT0Y67LAUBVA4T
 Arbiter (público) =
 GC27X2AXTFFF7UNDMBNQ55XVKRRTN07JPLUELEMPJJDQDA57G5TW3JUE

C.3 Contratos CCTP (Soroban, domínio 27)

Contrato	Mainnet
Token	
Messenger	CAE2G5Z77UP7GYPYGF0WFGW7C7J6I4YP2AFGSADRKQY62SYUFLPNFTXL
Minter	
Message	
Transmitter	CACMENFFJPJMSDAJQLX4R7K3SFZIW2LJSE3R2UMLGSHWFHS353FVXAZV
CCTP	
Forwarder	CBZL2IH7F6BIDAA3WBNXYKIXSATJGMSW7K5P5MJ6STX5RXN47TZJDF5T

Chains de origem suportadas (CCTP): Ethereum (domínio 0), Arbitrum (3), Solana (5), Base (6), Polygon PoS (7), Sui (8).

C.4 Stack tecnológico

Camada	Tecnologia
Escrow L1	Stellar setOptions + Multisig 2-de-3 + TimeBounds
SDK	@stellar/stellar-sdk v15 · @stellar/freighter-api
CCTP inbound	Soroban (forwarder) + Circle Iris V2
Backend	Node.js 20+ / TypeScript / Express
Banco de dados	MongoDB / Mongoose
Frontend	React 19 / Next.js 15 (Vercel)
Cron	JobService (runExpiredEscrows)
Testes	Jest — 409 testes em 23 suites

*NIDO — Plataforma B2B de campanhas com trustless escrow sobre Stellar L1.
Programa 37 Graus — NearX/SDF · Track A · 2026 · Mainnet em produção.*